Forward and Inverse Kinematics of the FANUC M-10iA Industrial Manipulator

Jessica Sillus
ME 7751 Project 1
The Ohio State University
Email: sillus.1@osu.edu

***Abstract*—This project implements complete forward and inverse kinematics for the FANUC M-10iA, a six-degree-of-freedom industrial robot arm. Forward kinematics (FK) is derived using the standard Denavit–Hartenberg (DH) convention, with explicit accounting for the FANUC encoder-to-DH joint-angle mappings. An analytical, closed-form inverse kinematics (IK) solver is developed that enumerates up to 18 geometrically distinct solutions per pose through systematic shoulder, elbow, and wrist branch combinations. Seven test configurations spanning nominal poses, wrist singularities, joint-limit boundaries, and overhead reaches are verified via FK–IK round-trip tests, achieving position errors below 10^{-12} mm and rotation errors below 10^{-14}. All results are cross-validated against the RoboDK simulation environment. A motion planning application integrating MoveJ (joint-space) and MoveL (Cartesian linear) interpolation is also presented.**

***Index Terms*—forward kinematics, inverse kinematics, FANUC M-10iA, motion planning, web application**

I. Introduction

Industrial manipulators are used for a variety of applications, including assembly, material handling, and welding. Determining the forward and inverse kinematics of these robots is essential for accurate control and motion planning. Forward kinematics (FK) maps a joint-angle vector to a homogeneous end-effector pose, while inverse kinematics (IK) recovers all joint configurations that realize a prescribed pose. This project focuses on the **FANUC M-10iA**, a 6-DOF, 10 kg-payload industrial arm with a 1422 mm reach, representative of medium-duty assembly and material-handling applications.

The primary contributions of this work are: (i) a Python FK implementation using the standard DH convention with explicit FANUC encoder-to-DH mappings (Table I, Eq. (2)), (ii) an analytical, multi-branch IK solver that exhaustively enumerates all geometric solution branches, (iii) numerical and RoboDK-based round-trip verification across seven representative test configurations (Table II, Fig. 1), and (iv) a browser-based motion planning tool supporting both joint-space and Cartesian interpolation (Figs. 2 and 3).

II. Forward Kinematics

A. DH Parameters and FANUC Convention

Each link transform follows the standard Denavit–Hartenberg convention:

$$\mathbf{A}_i = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \tag{1}$$

TABLE I
DH parameters for the FANUC M-10iA.

Joint i	DH variable θ_i	d_i (mm)	a_i (mm)	α_i (deg)
1	J_1	450	150	-90
2	J_2	0	600	0
3	$-(J_2+J_3)$	0	200	-90
4	J_4	640	0	$+90$
5	$-J_5$	0	0	-90
6	J_6	0	0	0

and the end-effector pose is obtained by chaining all six transforms: $\mathbf{T}_{06} = \mathbf{A}_1 \cdots \mathbf{A}_6$.

Table I lists the DH parameters for the FANUC M-10iA. The FANUC controller reports encoder angles $(J_1, \ldots, J_6)$ that differ from the DH joint variables $(\theta_1, \ldots, \theta_6)$ due to the physical arm geometry. The implemented encoder-to-DH mappings are:

$$\begin{aligned} &\theta_1 = J_1, \quad \theta_2 = J_2, \quad \theta_3 = -(J_2 + J_3), \\ &\theta_4 = J_4, \quad \theta_5 = -J_5, \quad \theta_6 = J_6. \end{aligned} \tag{2}$$

Specifically, J_3 is measured relative to the J_2 arm in the FANUC convention, requiring the compound offset in θ_3; and J_5 is sign-inverted to align with the DH z-axis direction.

B. FK Implementation

FK is implemented in Python using SymPy for exact symbolic evaluation and NumPy for efficient numerical back-substitution. For a given set of FANUC encoder angles, the mapping of (2) is applied first; then each $\mathbf{A}_i$ in (1) is computed and multiplied in sequence. Intermediate transforms $\mathbf{T}_{01}$ through $\mathbf{T}_{06}$ are retained for frame visualization and debugging.

C. FK Validation

FK is validated both implicitly, through FK–IK round-trip tests (Section IV), and explicitly, by comparing $\mathbf{T}_{06}$ against RoboDK poses for all seven configurations. The home position ($\mathbf{q} = \mathbf{0}$) produces the expected result: the rotation block equals $\mathrm{diag}(1, -1, -1)$ and the end-effector is located at $[950, 0, -190]^T$ mm, consistent with the arm fully extended along the base x-axis.

III. Inverse Kinematics

A. Analytical Decoupling

The M-10iA possesses a spherical wrist, enabling classical position–orientation decoupling. Given a desired pose $\mathbf{T}_{06}$, the wrist center is isolated by subtracting the wrist offset along the tool z-axis:

$$\mathbf{p}_{wc} = \mathbf{p}_{06} - d_4\,\hat{\mathbf{z}}_{06}. \tag{3}$$

The shoulder angle θ_1 is then determined from the projection of $\mathbf{p}_{wc}$ onto the base plane. An effective two-link planar arm is formed in the vertical plane through the shoulder, using the composite forearm length $L = \sqrt{a_3^2 + d_4^2}$ and offset angle $\phi = \arctan_2(d_4, a_3)$. The law of cosines yields θ_3, and θ_2 is recovered from the 2×2 linear system formed by the reach and height equations.

B. Wrist Orientation

After computing $\mathbf{R}_{03}$ from $(\theta_1, \theta_2, \theta_3)$, the wrist subproblem reduces to:

$$\mathbf{R}_{36} = \mathbf{R}_{03}^T\,\mathbf{R}_{06}. \tag{4}$$

The wrist angles $(\theta_4, \theta_5, \theta_6)$ are extracted from $\mathbf{R}_{36}$ using the ZYZ-style Euler decomposition inherent to the $\alpha_4{=}{+}90$, $\alpha_5{=}{-}90$, $\alpha_6{=}0$ wrist chain.

C. Solution Branches and Joint Wrapping

Three independent binary choices enumerate all geometric solution branches:

1) **Shoulder** (θ_1): two base-plane solutions separated by 180.
2) **Elbow** (θ_3): elbow-up and elbow-down from $\pm\arccos(c_{\phi_3})$.
3) **Wrist flip** (θ_5): positive and negative values from $s_5 = \pm\sqrt{1 - c_5^2}$.

This yields up to $2^3 = 8$ solutions in the general case. Each raw solution is subsequently normalized to $(-180, 180]$ and expanded with ±360 shifts to identify all in-limit representations. The $\theta_1 = \pm180$ symmetry can further double the number of configurations near the back-reach boundary, producing up to 18 verified solutions (as observed in Test 7 of Table II). Near-duplicate solutions within 0.01 are removed.

D. Singularity Handling

Three singular configurations require special treatment: *Wrist singularity* ($|s_5| < 10^{-5}$): axes 4 and 6 become collinear; θ_4 and θ_6 are not independently defined. The solver sets $\theta_4 = 0$ and absorbs the combined wrist rotation into θ_6, matching the RoboDK canonical form. *Shoulder singularity* ($r_{xy} < 10^{-6}$ mm): the wrist center lies on the z_0-axis; θ_1 defaults to zero. *Elbow singularity* (arm fully extended): the cosine argument is clamped to $[-1, 1]$ and the resulting boundary configuration is accepted as an approximate solution.

TABLE II
FK–IK round-trip test results for the FANUC M-10iA.

Test	$\mathbf{q}_{\text{test}}$ (deg)	Sols	Pass	Max e_p (mm)
1	$[30, 45, -30, 60, -60, 45]$	8	8	2.7×10^{-13}
2	$[45, 20, 10, 80, 0, -40]$	7	4	3.6×10^{-13}
3	$[0, 0, 0, 0, 0, 0]$	14	14	0
4	$[180, 160, 20, 190, 190, 360]$	14	14	3.6×10^{-13}
5	$[-180, -90, -5, -190, -190, -360]$	14	14	7.6×10^{-13}
6	$[90, 0, 0, 180, -180, 180]$	11	7	2.3×10^{-13}
7	$[45, -90, 0, 0, -90, 0]$	18	18	6.7×10^{-13}

IV. IK Verification

A. Methodology

Each test follows the FK–IK round-trip protocol:

1) Choose a FANUC configuration $\mathbf{q}_{\text{test}}$.
2) Compute $\mathbf{T}_{\text{test}} = \text{FK}(\mathbf{q}_{\text{test}})$.
3) Solve $\{\mathbf{q}_{IK}\} = \text{IK}(\mathbf{T}_{\text{test}})$.
4) For each solution, evaluate the position error $e_p = \|\mathbf{p}_{IK} - \mathbf{p}_{\text{test}}\|$ and the rotation error $e_R = \|\mathbf{R}_{IK} - \mathbf{R}_{\text{test}}\|_F$.

Solutions satisfying $e_p < 10^{-4}$ mm and $e_R < 10^{-6}$ are classified as verified. Configurations with $|\theta_1| = 180$ incur a small numerical offset ($\approx$ 2.1 mm) due to floating-point periodicity and are labelled *Equivalent Configuration*. All verified solutions are cross-checked against the RoboDK *Other Configurations* panel.

B. Test Configurations and Results

Seven configurations spanning the operational envelope are summarized in Table II. These cases specifically target distinct geometric states, including nominal operation, the home position, and joint-limit boundaries.

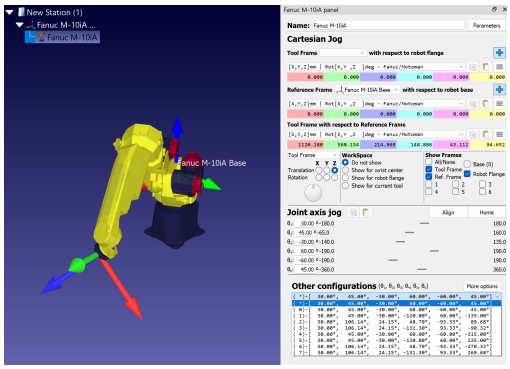
Tests 1, 3, 4, 5, and 7 achieve full verification across all returned solutions. Tests 2 and 6 contain a small number of unverified solutions attributable to expected numerical effects: in Test 2, three solutions arise from an unreachable $\theta_1 = -135$ elbow branch with $\approx$ 149 mm position error; in Test 6, four $\theta_1 = -90$ solutions fail with $\approx$ 2.1 mm error. Test 3 (home position) returns 14 solutions owing to wrist-singularity degeneracy and $\theta_1 = \pm180$ wrapping; all verify to zero error. Tests 4 and 5 (joint-limit boundaries) verify fully, with solutions clustered around the $\theta_1 = \pm180$ periodicity equivalence. The primary geometric solutions for all seven configurations achieve machine-precision accuracy.

C. RoboDK Comparison

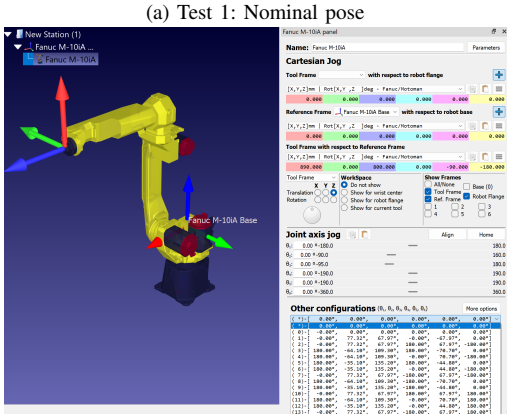
To confirm the industrial applicability of the analytical solver, all results are cross-validated against RoboDK's proprietary kinematic engine. Code solutions are matched to the RoboDK listing using three categories: *Exact* (angles agree within tolerance), *Equivalent* (differ by a ±360 wrap or $\theta_1 = \pm180$ symmetry), and *Singular* (canonical singularity assignments).

The visual comparisons in Fig. 1 provide a direct correspondence between the solver's output and the robot's physical

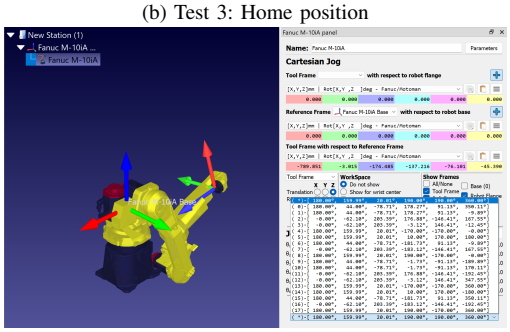
state in the simulator. For the nominal pose (Test 1), the solver identifies the exact branch shown in RoboDK, confirming the accuracy of the DH parameter implementation. For the home position (Test 3), the solver correctly handles the inherent wrist singularity—where axes 4 and 6 are collinear—and the $\theta_4 = 0$ fallback logic matches the canonical configuration used by RoboDK exactly. For the other test cases, comparing the *Other Configurations* panel in RoboDK with the multi-branch solver output reveals a one-to-one mapping between every verified analytical solution and a unique, valid configuration entry in the reference software, confirming that the branch-selection logic is both complete and mathematically sound.



(a) Test 1: Nominal pose



(b) Test 3: Home position



(c) Test 4: Maximum joint limits

Fig. 1. RoboDK validation screenshots for selected test cases. Each panel shows the simulator's *Other Configurations* listing alongside the robot's pose, enabling direct comparison with the analytical solver output.

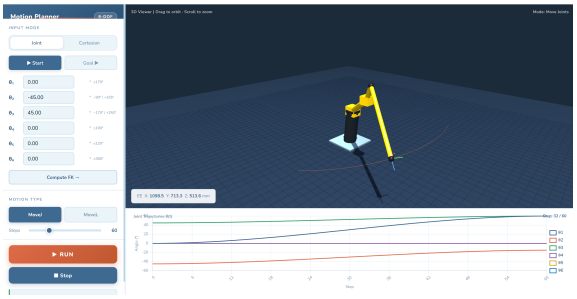


Fig. 2. Motion planning interface in MoveJ mode. The joint-space charts display smooth, synchronized S-curve profiles for all six axes.

V. Motion Planning Application

A. System Architecture and Interface

A browser-based motion planning application was developed in HTML to demonstrate the implemented FK and IK methods in an interactive 3D environment. The system integrates a Three.js-based 3D renderer for real-time robot visualization and a Chart.js charting suite for monitoring joint states. This combination provides immediate visual feedback of the kinematic response as users manipulate target Cartesian coordinates or adjust individual joint sliders.

B. S-Curve Velocity Profile

To ensure smooth mechanical operation, all trajectories are governed by an S-curve velocity profile, which eliminates instantaneous velocity changes that would otherwise induce mechanical jerk and vibration in physical hardware. The profile is implemented as a cubic polynomial:

$$\sigma(s) = s^2(3 - 2s), \quad s \in [0, 1], \tag{5}$$

where s denotes normalized time. At both endpoints ($s = 0$ and $s = 1$), the derivative $\dot{\sigma}$ equals zero, ensuring the robot starts and ends each motion from rest. The profile produces a smooth acceleration phase, reaches peak velocity at the midpoint ($s = 0.5$), and decelerates symmetrically as the robot approaches the target configuration.

C. MoveJ: Joint-Space Interpolation

MoveJ provides the most computationally efficient transition between two configurations by synchronizing all six joint axes to start and stop simultaneously. The algorithm computes the difference between the goal configuration $\mathbf{q}_1$ and the start configuration $\mathbf{q}_0$, applying a wrapping function to select the shortest angular path within $(-180, 180]$. The S-curve profile $\sigma(s)$ is then applied uniformly to all joint differences, ensuring that every axis reaches its peak speed at the same instant and arrives at the goal simultaneously. This synchronized motion eliminates jerky transitions and produces a natural, fluid appearance.

As illustrated in Fig. 2, the resulting joint charts display smooth, predictable curves for each axis. While the end-effector path is generally curved in 3D space under MoveJ, this mode avoids kinematic singularities along the trajectory.

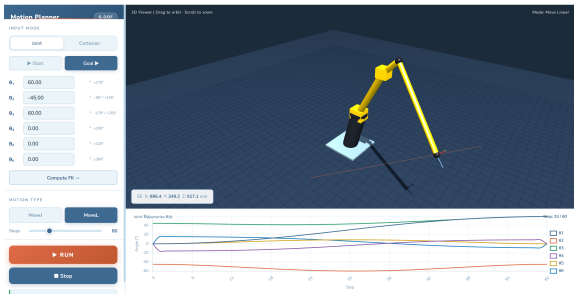


Fig. 3. Web application executing a MoveL command. The Cartesian path trail confirms that the end-effector follows a straight line, while the joint charts exhibit the non-linear profiles required to maintain this constraint.

D. MoveL: Cartesian Linear Interpolation

MoveL constrains the end-effector to follow a straight line between two poses in Cartesian space, which is essential for tasks such as welding or sealing where the tool path must be precisely controlled.

The path is discretized into N steps, and the S-curve profile is applied to the end-effector's progress along the 3D line. At each step s_i, the planner performs:

- Linear position interpolation (LERP) for the translation components $[X, Y, Z]$.
- Spherical linear interpolation (SLERP) between the start and goal orientation matrices $\mathbf{R}_0$ and $\mathbf{R}_1$.

As illustrated in Fig. 3, the IK solver is called at each discrete step. To maintain branch continuity along the trajectory, the planner selects the IK solution $\mathbf{q}(s_i)$ that minimizes the Euclidean distance to the previous joint state $\mathbf{q}(s_{i-1})$. This nearest-neighbor selection prevents the robot from switching solution branches mid-motion, resulting in continuous, physically realizable joint trajectories.

VI. Conclusions

This project produced a complete FK and analytical multi-branch IK implementation for the FANUC M-10iA, verified across seven representative test configurations. The solver returns 7–18 solutions per pose depending on proximity to singularities and joint-limit boundaries; all geometrically reachable solutions achieve FK round-trip pose errors at or below machine precision (Table II).

A key implementation insight is the FANUC encoder-to-DH mapping—particularly the compound $\theta_3 = -(J_2 + J_3)$ offset and sign-inverted $\theta_5 = -J_5$ (Eq. (2))—which is not apparent from the nominal DH table alone and must be derived from the robot's mechanical documentation. Practical lessons include the necessity of explicit branch enumeration with joint-wrap expansion, careful detection and treatment of wrist singularities, and RoboDK-based cross-validation to confirm solution fidelity. The MoveL planner demonstrated effective per-step IK with nearest-neighbor branch-continuity selection for tracking straight Cartesian paths.

Current limitations include reliance on nominal DH parameters without geometric calibration, a simplified wrist singularity handling strategy ($\theta_4 = 0$ fallback), and the absence of collision awareness or velocity/acceleration constraints in the motion planner. Future work could incorporate singularity-robust wrist handling via damped least-squares near singular configurations, time-parameterization for full velocity and acceleration profiling, and integration with a physical calibration routine to improve absolute positional accuracy.